

Attention, computed by hand

An embedding gives a word the same vector wherever it appears. But "bank" by a river and "bank" with a loan are not the same word in use. Attention lets every token look at every other token and build a new vector from what it finds. Today you compute one attention head on three tokens, with integers, on paper.

Unit 5 · Language

80 minutes

Paper exercise in class

Calculator for exponentials

The problem

Q, K, V

By hand

The transformer

Paper exercise

Before Friday

PART 1 · 10 MINUTES

Context changes meaning

1

Friday's vectors are fixed per word. In "*the clinic needs medicine*" we want the representation of *needs* to carry something of *clinic* and *medicine*. Session 13's network processed each example alone. What is missing is a way for positions in one sequence to **exchange information**, with the amount decided by the content itself.

ATTENTION IS A WEIGHTED AVERAGE

For each token, compute a set of weights over all the tokens — how much to *attend* to each — then take the weighted average of their vectors. The weights come from dot products, so tokens whose vectors align attend to each other more.

Queries, keys and values

Three roles, three learned matrices.

2



Query

What this token is looking for. $q = x \cdot W_Q$



Key

What this token offers to be found by. $k = x \cdot W_K$



Value

What this token hands over when attended to. $v = x \cdot W_V$

SCALED DOT-PRODUCT ATTENTION

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d}}\right) \cdot V$$

Score every query against every key, scale by $\sqrt{d}$, turn each row of scores into weights with a softmax, then mix the values.

Every piece of this is something you already own. $Q \cdot K^T$ is session 8's matrix multiply. The softmax is session 11's sigmoid generalised to many options. The weighted sum is another matrix multiply. Dividing by $\sqrt{d}$ keeps scores from growing with the vector length and pushing the softmax into its flat, saturated ends — session 19's problem again.

One head, three tokens

clinic → needs → medicine, with $d = 2$.

3

Token	Query	Key	Value
clinic	[1, 0]	[1, 0]	[2, 0]
needs	[1, 1]	[1, 1]	[1, 1]
medicine	[0, 1]	[0, 2]	[0, 3]

- Scores for *clinic*.** Its query [1, 0] against the three keys: $1 \cdot 1 + 0 \cdot 0 = 1$, $1 \cdot 1 + 0 \cdot 1 = 1$, $1 \cdot 0 + 0 \cdot 2 = 0$.
- Scale by $\sqrt{2} \approx 1.414$.** 0.707, 0.707, 0.
- Softmax.** $e^{0.707} \approx 2.03$ twice and $e^0 = 1$; the total is 5.06; the weights are **0.401, 0.401, 0.198**. They add to 1.
- Mix the values.** $0.401 \cdot [2, 0] + 0.401 \cdot [1, 1] + 0.198 \cdot [0, 3] = [1.203, 0.994]$. The new vector for *clinic* now contains part of *needs* and part of *medicine*.

THE FULL ATTENTION WEIGHTS — EVERY ROW SUMS TO 1

		clinic	needs	medicine
query	clinic	0.401	0.401	0.198
	needs	0.198	0.401	0.401
	medicine	0.140	0.284	0.576

attends to (key) →

medicine puts more than half its attention on itself — 0.576 — because its key [0, 2] is twice as long in the direction its own query [0, 1] points. Nothing *decided* that. A dot product was larger, so a score was larger, so a weight was larger.

The prediction question

Change the key for *medicine* from [0, 2] to [2, 0]. Before any arithmetic, predict which rows of the weights change, and which way their largest weight moves. Then open.

The key now points along the first axis, so it aligns with queries that have a first component. **clinic** ([1, 0]) now scores *medicine* highest: its row becomes [0.248, 0.248, 0.503]. **medicine** ([0, 1]) no longer aligns with its own key and shifts its largest weight to *needs*: [0.248, 0.503, 0.248]. **needs** ([1, 1]) scores 2 against the new key, exactly as before, so its row does not change at all.

PART 4 · 15 MINUTES

From one head to a transformer — recognised, not assembled

4

What you computed is one attention head. A transformer block, named part by part from the lecture diagram:

Part	What it does	Where you have met the idea
Token embeddings + positions	vectors for tokens, plus information about order — attention alone ignores order	session 24
Multi-head attention	several heads in parallel, each with its own W_Q, W_K, W_V , each free to track a different relationship	today, many times
Causal mask	in a text generator, a token may only attend to earlier tokens — no peeking at the answer	session 20's leakage, inside a model
Feed-forward layer	a small two-layer network applied to each position	session 13
Residual connections + normalisation	add each block's input back to its output; keep scales stable so deep stacks train	session 19

WHY WE STOP HERE

A full transformer is these blocks stacked a dozen to a hundred times, with learned matrices of millions of numbers. Building one does not fit in eighty minutes and would teach little beyond today. **One head on three tokens is the idea; the rest is scale.** On Friday you use a transformer somebody else built and trained.

PAPER EXERCISE · 25 MINUTES

One attention head, by hand

5

`CSC4315_S25_Paper_One_Attention_Head` : rows for *clinic* and *needs* with every intermediate number, the reading question, and the changed key — prediction first.

Before Friday

1. **Without notes:** query, key, value, why $\sqrt{d}$, and what the softmax does to a row of scores.
2. **Start Lab 17's first cell** before class: it downloads a small language model (about 350 MB).



Session 26 — Friday 04 December, 11:30

Using a language model you did not train: sampling, temperature, and fine-tuning it until you can hear its voice change.